

PowerModels Project Analysis v3

Generated: 2026-03-08 Sources:

- Implementation vault: scan-data.json (2026-03-08 rescan), 10 agent-generated architecture docs, 6 artifact docs
- Planning vault: 23 facets × 11 audiences, 253 intersections
- GitHub: 317 issues, 240 merged PRs, 9-month window (event-systems/powermodels)
- **New in v3:** Accountant workflow proposal (5 epics, 15 issues), process manager infrastructure, persisted read model infrastructure, corrected data-usage metrics

1. Codebase Inventory

Scan Summary (2026-03-08)

Metric	Value
Aggregates	27 production, 2 test-only
Commands	249
Events	443 (1.78× command count)
Handler Services	22
Read Models	80 (46 ReadModelBase + 34 TransientSubscriber)
Test Files	154
Bounded Contexts	7 (ModelServer, ModelServer/AccountingSystem, SpreadsheetAdapter, UIBehavior, Excel, TeamModelMgmt, App)

Aggregate Size Distribution

Tier	Aggregates	Lines Range	Notes
God	ServerFinancialModel	3,557	87 Apply methods, 31 Register calls, 166 commands via handler
Large	ChartOfAccounts (540), DataTableMap (370), ListDataTableMap (369), ManualTable (365), TasklistItem (340), ManualTableMap (306)	300–540	Active development targets
Medium	DataSource (271), ClientWorkspace (247), Counterparty (224), CustomStatementTxType (179), DataTableDefinition (176), ReferenceDataSeries (172), EntrySet (169)	140–280	Stable
Small	ManagedFinancialModel (125), AccountBalance (106), FileStore (98), Product (90), UserDefinedWorksheets (69), DataSourceMapping (61)	60–125	Lean

Minimal	Vendor (49), Customer (46), Contractor (36), Employee (36), AccountingSystem (29)	29–49	Shells or single-purpose
---------	---	-------	--------------------------

Read Model Subscription Topology

ReadModelBase RMs subscribe to specific aggregate category streams via `Start<Tagg>()` . They are **not** on `$All` .

Stream	Subscribers	Startup Replay Cost
\$ce-ServerFinancialModel	9 — ModelTemplateRm, FinancialModelRm, FinancialModelListRm, ModelListRm, ModelWorksheetTablesRm, DataTableFromTemplateRm, ModelVerificationRM, ModelsRm, SingleModelTaskMetricsRm	Hottest — 9× replay per SFM event
\$ce-DataSource	3 — DataSourceListRm, DataSourceRm, DataSourceUsageRm	Moderate
\$ce-ManagedFinancialModel	3 — DataSourcesRm, ModelsRm, TaskMetricsRm	Moderate
\$ce-ManualTable	3 — ModelWorksheetTablesRm, ManualDataTableRm, ManualTableRm	Moderate
21 other category streams	1–2 each	Low
5 event-type streams (\$et-*)	1 each	Low

34 TransientSubscriber RMs are bus-only (live events when UI is open). Zero startup cost.

Handler Concentration

Handler	Commands	% of Total	Aggregates Touched
FinancialModelService	166	67%	ServerFinancialModel
AccountingSystemService	34	14%	9 aggregates (AccountBalance, AccountingSystem, ChartOfAccounts, Counterparty, Customer, Vendor, Employee, Contractor, Product)
TasklistService	20	8%	TasklistItem
DataSourceService	12	5%	DataSource, DataElement
ManualTableService	12	5%	ManualTable, ManualTableMap
17 others	5 total	2%	Various

67% of all commands route through a single handler (FinancialModelService) to a single aggregate (ServerFinancialModel). This is the structural definition of God Aggregate coupling.

2. Architecture Assessment

What Works

Domain layer is clean. ModelServer has zero upstream dependencies. Aggregates follow ReactiveDomain conventions with Register/Apply patterns. 24 of 27 aggregates have idempotency tests. State guard tests present on 25 of 27.

ACL boundary is enforced. SpreadsheetAdapter is the sole bridge between UI and domain. UI projects cannot reference ModelServer. SpreadsheetContextBus decorates all messages with routing metadata.

Per-business isolation is strong. Each business has its own DataStore file. No cross-business event access. Workspace isolation via `ConcurrentDictionary<Guid, ClientWorkspaceContext>`.

Event-to-command ratio is healthy. 443 events / 249 commands = 1.78x. Events carry the right granularity.

Read/write separation is real. ModelServerReadContext and ModelServerWriteContext enforce separate wiring. Read models project from events; commands go through services.

Read Model Concentration

Top 5 read models by event subscription count:

Read Model	Events	Context	Risk
ModelTemplateRm	82	ModelServer	🔴 Subscribes to 19% of all event types
FinancialModelTablesRm	54	UIBehavior	🟡 Heavy UI-layer subscriber
FinancialModelRm	49	SpreadsheetAdapter	🟡 ACL-layer bridge, 4 defect PRs
WorkspaceTasklistRm	21	SpreadsheetAdapter	🟢 Reasonable
SingleClientTasklistRm	21	SpreadsheetAdapter	🟢 Reasonable

ModelTemplateRm processing 82 events (nearly 1 in 5 event types) is the highest-risk single RM. It also has 2 defect PRs.

What Doesn't Work

No replay safety net. 0/27 aggregates have RestoreFromEvents tests. If any Apply method has a bug that doesn't manifest until replay, it will silently corrupt aggregate state when the business DataStore is reopened.

No explicit sagas. 7 identified workflow chains, all coordinated implicitly through handler services. No compensation logic, no timeout handling, no workflow visibility.

Startup replay from position 0. 46 ReadModelBase RMs replay their category streams from the beginning every time a business is opened. No checkpoints, no persisted state — all RM state is in-memory only and lost on restart.

Full stream replay per GetById. Every `ICorrelatedRepository.GetById()` call replays the entire aggregate stream. ServerFinancialModel (the most frequently loaded aggregate) has the largest stream per business.

4 missing source constructors. ServerFinancialModel, ReferenceDataSeries, DataSourceMapping, AccountingSystem — these aggregates cannot properly participate in correlation tracking.

3. Domain Gaps (from Accountant Workflow Proposal)

The `small-accountant-flow-proposal.md` identifies what must be built for the accounting workflow to be complete. Cross-referencing with the codebase scan reveals:

Missing Aggregates

Aggregate	Purpose	Dependencies	Priority
Journal	First-class journal entries with balanced debits/credits, compensating entries, period-aware validation	AccountingSystem (fiscal year)	CRITICAL — every other accounting feature depends on it
ReconciliationState	Bank reconciliation tracking per account per period	Journal (posting adjustments)	HIGH
PeriodClose	Monthly close workflow, period locking (soft/hard)	AccountingSystem (fiscal year), Journal (entry validation)	HIGH

Missing Domain Concepts on Existing Aggregates

Aggregate	Missing	Current State
AccountingSystem (29 lines)	Entity type (Sole Prop, LLC, S-Corp, etc.), industry context, fiscal year, close policy	Contains only <code>AccountingSystemCreated</code> event
ChartOfAccounts (540 lines)	Industry template seeding, contra account UI workflow	Accounts exist, hierarchy exists; no industry-driven templates
RuleStep	Per-entity configurable rules (currently hardcoded keyword-matching only)	<code>RuleStep.cs</code> — hardcoded bank fees, interest, transfers

Missing Services

Service	Purpose
Entry Patterns Library	Pre-defined journal entry templates (purchase, revenue, payroll, depreciation, etc.)
GL Report Generation	Trial balance, income statement, balance sheet — currently only via QuickBooks import
Industry CoA Template Engine	Maps (entity type + industry + basis) → minimum viable CoA
Confidence Threshold Enforcement	85/15 auto-approve vs review boundary for classification

Codebase Status by Epic

Epic	Status	Exists	Missing
------	--------	--------	---------

1. Company Setup	PARTIAL (3/6)	AccountingSystem, ChartOfAccounts, business creation chain	Entity type, industry, fiscal year, close policy
2. CoA Templates	MOSTLY COMPLETE (5/7)	Account hierarchy, group sets, standard chart, contra flag	Industry templates, template seeding command
3. Journal & Ledger	PARTIAL (6/10)	EntrySet (manual entries), validation engine, reconciliation pipeline	Journal aggregate, entry patterns, GL reports, ledger RM
4. Classification & Rules	MOSTLY COMPLETE (6/8)	AIStep (300+ lines), review workflow, confidence tracking	Per-entity configurable rules, 85/15 threshold enforcement
5. Reconciliation & Close	PARTIAL (5/7)	Reconciliation pipeline (12+ steps), task aggregate	ReconciliationState aggregate, PeriodClose aggregate, adjustment patterns

Impact on Existing Architecture

Adding 3 new aggregates, extending AccountingSystem, and building new read models will:

- Add ~3 new category streams (`$ce-Journal` , `$ce-ReconciliationState` , `$ce-PeriodClose`)
- Add new ReadModelBase RMs (Ledger RM at minimum) — each replays from position 0 on startup
- Add new commands to AccountingSystemService (entity type, industry, fiscal year, close policy)
- Require new saga coordination: Journal → Reconciliation → PeriodClose → Reports

Without addressing the startup replay and saga coordination gaps first, every new aggregate and RM makes both problems worse.

4. Infrastructure Readiness

Available: Process Manager Base (Not Yet Integrated)

Located in `process-manager/Infrastructure/` :

ProcessAggregateBase<TState> — Event-sourced state machine:

- Generic state enum tracking with `State` property
- State-based handler DSL: `InState(State.Running).On<SomeEvent>(handler)`
- Command accumulation: `IssueCommand()` / `TakeCommands()` → `CommandsIssued` event
- Step timeout: `CreateStepTimeout(TimeSpan)` with staleness detection (wrong state/version auto-ignored)
- Terminal state definition via `TerminalStates` property
- `IsRunning` computed from non-default, non-terminal state

ProcessManagerBase<TProcessAgg, TState> — Read model coordinator:

- Extends `ReadModelBase` — subscribes to category streams via `Start<TAgg>()`
- NullBus replay safety: `ActiveBus` starts as `NullBus` , switches to live bus via `GoLive()`
- `LoadOrCreate(processId)` → returns `(Aggregate, IsNew)` tuple
- `SaveAndTakeCommands(aggregate)` → saves + extracts pending commands

- `DispatchCommands(commands)` → publishes to `ActiveBus`
- `OnGoLive()` hook for startup recovery of in-flight processes
- `StepTimedOut` routing by `ProcessTypeName`

Supporting:

- `CategoryStreamWarmup` — ESDB `$ce-*` stream warmup utility
- `EventVersioning` + `VersionedEventSerializer` — event schema evolution with `[EventVersion]` attribute and upcaster chain

Integration pattern:

```
Domain event → ProcessManager.Handle(event) → LoadOrCreate(processId)
→ aggregate.When(event) → state transition + IssueCommand()
→ SaveAndTakeCommands(aggregate) → DispatchCommands(commands) → bus
```

Available: Persisted Read Model Base (Not Yet Integrated)

Located in `process-manager/read-model-caching-reference/` :

IReadModelStore<TKey, TModel> — Generic persistence interface:

- `Insert` / `Update` / `Upsert` / `GetById` / `GetAll` / `GetWhere`
- Transactional overloads accepting `DbConnection` + `DbTransaction`
- SQL-level `GetWhere(string whereClause, object parameters)` for DB-side filtering

Implementations:

- `InMemoryReadModelStore` — `ConcurrentDictionary` -backed, hot-path queries
- `DapperReadModelStore` — PostgreSQL via Dapper, POCO mapping with `[Table]` / `[Column]` attributes, auto snake_case conversion

ICheckpointStore — Projection position tracking:

- `GetCheckpoint(projectionName)` → `long?`
- `SaveCheckpoint(projectionName, position)` with transactional overload
- `InMemoryCheckpointStore` — for testing
- `PostgresCheckpointStore` — `projection_checkpoints` table with `INSERT ... ON CONFLICT DO UPDATE`

ProjectionSchema — DDL management:

- `CREATE TABLE IF NOT EXISTS` for all projection tables + checkpoint table
- `InitializeAsync(NpgsqlDataSource)` — safe to call on every startup

Integration pattern:

```
ReadModelBase constructor accepts optional ICheckpointStore
→ On startup: GetCheckpoint(name) → resume from last position (not position 0)
→ On each event: update IReadModelStore + SaveCheckpoint atomically
→ Two-tier: InMemoryReadModelStore (hot) + DapperReadModelStore (persisted)
```

Gap: Neither Infrastructure Is Integrated Into PowerModels

Both the process manager and read model caching exist as reference implementations from the Greylock project. Neither has been wired into the PowerModels codebase. Integration requires:

1. **NuGet or project reference** to the infrastructure assemblies
2. **PostgreSQL database** added to the per-business or firm-level topology (currently only DataStore + SQL Server for Identity)
3. **ReadModelBase constructor changes** to accept `ICheckpointStore`
4. **Migration of existing RMs** from in-memory-only to two-tier storage

5. Risk Analysis

Risk A: God Aggregate — ServerFinancialModel

Metric	Value	Source
Lines	3,557	scan-data.json
Apply methods	87 (vs 31 Register calls)	scan-data.json
Commands via handler	166 of 249 (67%)	message-map.md
PRs touching it (9mo)	9 — all large (26-94 files each)	feature-provenance.md
Defects (via read models)	7 (FinancialModelRm: 4, ModelTemplateRm: 2, FinancialModelService: 1)	defect-analysis.md
Category stream subscribers	9 (hottest stream)	read-models.md
Missing source ctor	Yes	aggregates.md
RestoreFromEvents test	No	test-coverage.md

Why it compounds: 9 RMs replay `Ice-ServerFinancialModel1` on every startup. Every feature that touches SFM adds events to the largest stream. The 87:31 Apply:Register ratio indicates overloaded event handling — Apply methods handle variants the Register calls don't distinguish.

Accountant workflow impact: The proposal explicitly avoids expanding SFM. New features (Journal, Reconciliation, PeriodClose) are separate aggregates. But the Financial Model Table Mapping saga still couples through SFM.

Risk B: All Sagas Are Implicit — Infrastructure Exists But Not Integrated

Metric	Value
Identified workflow chains	7
Explicit process managers in PowerModels	0
Process manager infrastructure ready	Yes — <code>ProcessAggregateBase<TState></code> + <code>ProcessManagerBase<TProcessAgg, TState></code>
Domain-crossing PRs (9mo)	38
Highest-risk saga	Reconciliation (5 contexts, 5 aggregates, 19 defects)

Accountant workflow new sagas	3+ (Journal→Reconciliation→PeriodClose, Business Setup extended, Classification→Rules)
-------------------------------	--

Why it compounds: The accountant workflow proposal adds new cross-aggregate coordination (Journal posting → Reconciliation validation → PeriodClose enforcement → GL reporting). Implementing these as implicit sagas repeats the pattern that generated 29% of all defects. The process manager infrastructure exists to prevent this but hasn't been integrated.

Risk C: SpreadsheetAdapter Is the Defect Funnel

Metric	Value
RMs in SpreadsheetAdapter	40 of 80 total
Defect rate	15% (highest of all contexts)
Defect-affected RMs in SpreadsheetAdapter	13 of 16 defect-hit RMs
Role	ACL bridge: domain events → UI-consumable projections

Why it compounds: The accountant workflow adds new domain events (JournalEntryPosted, PeriodClosed, ReconciliationCompleted, etc.). Each needs SpreadsheetAdapter RMs to project to the UI. Without RM persistence, adding RMs increases startup replay cost. Without reducing the existing 40 RMs, adding more increases the defect surface.

Risk D: Startup Performance Degrades With Every New RM

Metric	Current	After Accountant Workflow
ReadModelBase RMs	46	~50+ (Ledger RM, ReconciliationStateRm, PeriodCloseRm, ClassificationRulesRm)
Category streams	25	~28+ (Journal, ReconciliationState, PeriodClose)
RM state persistence	None — in-memory only	None unless checkpoint infrastructure is integrated
Startup cost	46 category stream replays from position 0	~50+ category stream replays from position 0

The RM checkpoint infrastructure (`ICheckpointStore` + `IReadModelStore`) directly addresses this but requires integration before new RMs are added.

The Compound Cycle

New accounting features (Journal, Reconciliation, PeriodClose)

- New aggregates → new category streams → new events
- New ReadModelBase RMs subscribing to those streams
- Longer startup replay (no checkpoints)
 - New cross-aggregate sagas (implicit, no compensation)
 - More SpreadsheetAdapter RMs (defect funnel)
 - More defects at the ACL boundary

→ God Aggregate still coupled via table mapping
 → (cycle repeats)

Breaking the cycle requires integrating the checkpoint and process manager infrastructure BEFORE building the accountant workflow features.

Aggregate Risk Matrix (Full Cross-Reference)

Aggregate	Lines	PRs (9mo)	Defects (via RM)	RestoreFromEvents	Source Ctor	T F
ServerFinancialModel	3,557	9	7	NO	NO	4
ChartOfAccounts	540	14	3 (ChartOfAccountsRm)	NO	Yes	4
TasklistItem	340	6	0	NO	Yes	7
DataTableMap	370	4	1 (DataTableMapRm)	NO	Yes	5
ListDataTableMap	369	4	1 (ListDataTableMapRm)	NO	Yes	4
CustomStatementTxType	179	4	0	NO	Yes	1
ManualTableMap	306	4	2 (ManualTableMapRm)	NO	Yes	4
DataSource	271	3	4 (DataSourceRm)	NO	Yes	8
AccountBalance	106	2	1 (AccountBalancesRm)	NO	Yes	8
ReferenceDataSeries	172	0	0	NO	NO	3
DataSourceMapping	61	0	0	NO	NO	3
AccountingSystem	29	0	0	NO	NO	4

--	--	--	--	--	--	--

Stable Aggregates (Zero PRs in 9 Months)

Aggregate	Lines	Context	Signal
ManualTable	365	ModelServer	✓ Mature — 11 Apply methods, well-tested
ClientWorkspace	247	ModelServer	✓ Mature — setup-only, rarely changed
ReferenceDataSeries	172	ModelServer	⚠ Missing source ctor — dormant risk
EntrySet	169	ModelServer	✓ Mature
ManagedFinancialModel	125	TeamModelMgmt	✓ Mature
TeamSettings	105	TeamModelMgmt	✓ Mature
UserDefinedWorksheets	69	ModelServer	✓ Mature
DataSourceMapping	61	ModelServer	⚠ Missing source ctor — dormant risk
AccountingSystem	29	ModelServer/AS	⚠ Missing source ctor — next expansion target

Stability validates that these aggregates have settled. The three with missing source ctors are dormant risks. AccountingSystem is about to exit stability — the accountant workflow proposal extends it with entity type, industry, and fiscal year.

6. Defect Patterns

By Root Cause (26 classified of 66 total)

Pattern	Count	Saga	Context	Root Cause
Transaction import corruption	8	Data Import Pipeline	SpreadsheetAdapter	PDF/CSV parsing edge cases
Read model state sync	6	All sagas	SpreadsheetAdapter	RM not updating — race conditions, missing handlers
UI component state	5	N/A	UIBehavior/WPF	WPF component state out of sync with RM
Account mapping errors	4	Reconciliation	SpreadsheetAdapter	Incorrect account resolution
Business setup workflow	3	Business Setup	ModelServer→SpreadsheetAdapter	Multi-step creation stuck

By Read Model (top defect-hit RMs)

RM	Defect PRs	Event Subscriptions	Stream
DataSourceRm	4	11	\$ce-DataSource
FinancialModelRm	4	49	\$ce-ServerFinancialModel
DataSourceListRm	3	10	\$ce-DataSource
ChartOfAccountsRm	3	4	\$ce-ChartOfAccounts
ModelTemplateRm	2	82	\$ce-ServerFinancialModel
ManualTableMapRm	2	14	\$ce-ManualTableMap

13 of 16 defect-affected RMs are in SpreadsheetAdapter. Defects don't hit aggregates directly — they surface in the projection layer.

Defect Rate by Milestone

Milestone	Features	Defects	Rate	Signal
Controlled Beta - u4	8	9	53%	🔴 Bug fix milestone
Shoebox RC1	12	7	37%	🟡 Early stabilization
Shoebox RC2	16	8	33%	🟡 Continued stabilization
Demo Update - March	17	8	32%	🟡 Active development
Shoebox RC5	25	10	29%	🟡 RC4 defects surfacing
Shoebox Public Beta 1	20	4	17%	🟢 Moderate
Shoebox RC4	55	4	7%	🟢 Best ratio — reconciliation rework was clean
Shoebox RC3	30	3	9%	🟢 Healthy

RC4 (the largest milestone at 55 features) had the lowest defect rate (7%). The reconciliation rework was well-executed. RC5's higher rate (29%) is mostly RC4 edge cases surfacing in production.

God Aggregate Defect Ecosystem

ServerFinancialModel itself: 0 direct defect PRs. But its RM ecosystem accounts for 7 defects (20% of 35 defect PRs): FinancialModelRm (4), ModelTemplateRm (2), FinancialModelService (1). The aggregate is well-tested; the complexity leaks through its 87 Apply methods into 9 subscribing RMs.

Development Velocity

Phase	Period	Issues	Character
Beta stabilization	Jun 2025	65	Backlog seeded
Shoebox buildout	Jul–Nov 2025	139	RC1→RC3, reconciliation, counterparties
Reconciliation rework	Dec 2025–Feb 2026	113	RC4 (55 features, 7% defect rate — cleanest)

Accounting phase 2	Mar 2026	11+	Opening balances, accountant workflow
--------------------	----------	-----	---------------------------------------

Transition point: The project is shifting from data import/reconciliation (Shoebox) to accounting workflows. ChartOfAccounts is now the most active aggregate (14 PRs in 9mo), while ServerFinancialModel is stabilizing (9 PRs). The accountant workflow proposal accelerates this shift.

7. Q-Analysis: Facet × Audience Resonance

Strongest Connections (Dimension 3+)

35 intersections have 3+ hooks — these are where the product story is strongest:

Facet	Dim-3+ Audiences	Signal
unit-economics	9 of 11 audiences	Universal — everyone cares about the business model
competitive-positioning	4 (investors, board, founding-team, CFOs)	Strong investor/governance story
reconciliation-elimination	4 (investors, board, founding-team, firm-partners)	Core value prop connects broadly. GitHub validates: RC4's 55-feature reconciliation rework, 19 defects (29% of total), 4 epics
shoebox-offering	4 (firm-partners, bookkeepers, founding-team, tax-preparers)	Strong practitioner resonance. GitHub validates: 12 Shoebox-labeled features, entire RC1→RC5 arc
event-sourced-architecture	2 (CFOs, founding-team)	Technical moat understood by few
drift-metric	2 (CFOs, founding-team)	Operational KPI resonates with financial users
future-ledger	2 (CFOs, founding-team)	Deterministic forecasting = CFO language

Weakest Connections (Dimension 0)

78 intersections have zero hooks:

Facet Group	Dim-0 Count	Why
Technical architecture facets (6)	63	Technical facets don't connect to business audiences
Business facets × data-usage-reviewer	15	Technical audience doesn't connect to business facets

This is structurally correct — dimension-0 gaps between technical and business facets confirm the taxonomy is properly separated.

The Communication Gap

- **Founding team** is the only audience connecting to *both* technical and business facets
- **Investors and board** connect strongly to business facets but weakly to technical ones

- **Practitioners** (firm-partners, bookkeepers, tax-preparers) connect to workflow facets but not architecture

Implication: The technical architecture story (event sourcing, isolation, data usage) should be translated into business terms for investors/board (competitive moat, scaling efficiency, infrastructure cost) rather than presented as technical detail. GitHub confirms: 0 of 229 features in 9 months were described in architectural terms — all are user stories.

Accountant Workflow Resonance

The accountant workflow proposal maps directly to the strongest facets:

- **reconciliation-elimination** — Epics 3 (Journal) and 5 (Reconciliation/Close) directly deliver this
- **shoebox-offering** — Epic 1 (Company Setup) and Epic 2 (CoA Templates) enable the onboarding promise
- **proven-completeness** — Epic 3 (GL Reports) and Epic 5 (Period Close) create the defensible output
- **professional-judgment-preservation** — Epic 4 (Classification Rules, 85/15 threshold) preserves CPA control

8. Test Gap Analysis

Metric	Value	Risk
RestoreFromEvents tests	0 / 27	🔴 Critical — replay corruption undetectable
Idempotency tests	24 / 27	🟢 Good — missing: DataSourceMapping, EntrySet, ReferenceDataSeries
State guard tests	25 / 27	🟢 Good — missing: ReferenceDataSeries, TeamSettings
Missing source constructors	4	🟡 ServerFinancialModel, ReferenceDataSeries, DataSourceMapping, AccountingSystem

The accountant workflow adds 3 new aggregates. Each **MUST** have:

- RestoreFromEvents test (project has zero precedent — must establish pattern first)
- Source constructor (`:base(source)`)
- Idempotency + state guard tests

Recommendation: Write the first RestoreFromEvents test for ServerFinancialModel before adding new aggregates. This establishes the pattern and validates the most-replayed aggregate.

9. Data Storage and Performance

Current Topology

Database	Scope	Technology	Domain Data
DataStore	Per business	Custom in-memory + Protobuf3 disk	All domain events
PowerModelsContext	Firm-wide	SQL Server + EF Core	Auth only (no domain data)
File System	Per business	Files on disk	Model workbooks, imported CSV/PDF

Startup Sequence

- 1. Read full PersistentAllStream file into in-memory DataStore (fixed cost per business)
- 2. Rebuild all streams (\$All, \$ce-*, \$et-*) from the loaded events
- 3. 46 ReadModelBase RMs each replay their category stream from position 0
- 4. TransientSubscriber RMs start listening on the bus (no replay)

Performance Scaling

Business Maturity	Events	Disk	Hottest Stream Startup (9 RMs × SFM events)
New	25-50	5-15 KB	Negligible
First year	3,000-8,000	0.6-1.6 MB	~5K × 9 = ~45K handler invocations
Mature (2-3 years)	15,000-30,000	3-6 MB	~15K × 9 = ~135K handler invocations
Heavy use	50,000+	10+ MB	~25K × 9 = ~225K+ handler invocations

After Checkpoint Integration

With ICheckpointStore , ReadModelBase RMs resume from their last position instead of position 0. A business that processed 15K events yesterday only replays new events today. Startup cost drops from O(total events × subscribers) to O(new events × subscribers).

10. Prioritized Actions

P0 — Before Adding New Aggregates

#	Action	Evidence	Effort
1	RestoreFromEvents test for ServerFinancialModel	0/27 tested, 87 Apply methods, 3,557 lines	2-3 days
2	Fix 4 missing source constructors	ServerFinancialModel, ReferenceDataSeries, DataSourceMapping, AccountingSystem	1 day
3	Integrate ICheckpointStore into ReadModelBase	46 RMs replay from position 0; infrastructure ready but not wired	1-2 weeks

P1 — Foundation for Accountant Workflow

#	Action	Evidence	Effort
4	Integrate ProcessManagerBase into PowerModels	7 implicit sagas, 0 explicit; accountant workflow adds 3+ new cross-aggregate workflows	1-2 weeks
5	Migrate top 9 RMs on \$ce-ServerFinancialModel to checkpoint + persistence	Hottest stream, 9× fan-out, most startup cost	2-3 weeks
6	Extend AccountingSystem with entity type, industry, fiscal year (Epic 1.1-1.3)	29-line shell, foundational for all accounting features	1-2 weeks

P2 — Accountant Workflow Implementation

#	Action	Evidence	Effort
7	Create Journal aggregate (Epic 3.1)	CRITICAL gap — every accounting feature depends on it	2-3 weeks
8	Explicit reconciliation ProcessManager	#1 defect generator (19/66), 5 contexts, no compensation	2-3 weeks
9	SpreadsheetAdapter RM audit	40 RMs, 15% defect rate, 13/16 defect-hit RMs	1-2 weeks
10	Extract table-mapping from ServerFinancialModel	20 extractable events, god-aggregate.md	1-2 weeks

P3 — Scaling and Quality

#	Action	Evidence	Effort
11	RestoreFromEvents tests for all 27 aggregates	0/27 tested — establish pattern from #1, then batch	1-2 weeks
12	ReconciliationState + PeriodClose aggregates (Epic 5.1-5.2)	Missing domain concepts, required for close workflow	2-3 weeks
13	Entry Patterns Library (Epic 3.2)	No journal gestures; needed for both manual entry and AI classification	1-2 weeks
14	GL Report Generation (Epic 3.3)	Only via QuickBooks import currently	2-3 weeks

11. Feature Trial Candidates

Trial A: God Aggregate Extraction (Analysis-Driven)

Extract DataTable mapping from ServerFinancialModel

Support	Evidence
Structural	god-aggregate.md: 20 extractable events, dedicated aggregates already exist
Historical	feature-provenance: 9 PRs changed 26-94 files each — cascade effect is measured
Defect	defect-analysis: 7 defects via FinancialModelRm/ModelTemplateRm
Saga	saga-catalog: Financial Model Mapping workflow shows exactly where coupling occurs
Q-analysis	event-sourced-architecture × founding-team = Dim-3+

Vault coverage: 5 of 9 architecture docs directly relevant. This is the best-instrumented change in the vault.

Trial B: Explicit Reconciliation Saga (Defect-Driven)

Extract reconciliation coordination into a dedicated ProcessManager

Support	Evidence
Defect	defect-analysis: 19 defects (29% of total), #1 defect pattern
Saga	saga-catalog: 5 contexts, 5 aggregates, implicit coordination, no compensation
Feature	feature-provenance: 4 epics, RC4's entire feature set
Scale	data-usage-and-storage: ~1,001 events per import × per-stream fan-out
Infrastructure	ProcessManagerBase ready — NullBus replay safety, LoadOrCreate, GoLive recovery
Q-analysis	reconciliation-elimination × 4 audiences = Dim-3+

Strongest defect justification of any trial. Also validates the process manager infrastructure — if the explicit saga reduces defect rate, it proves the infrastructure is production-ready.

Trial C: Accountant Workflow Foundation (Domain-Driven)

Extend AccountingSystem + Create Journal Aggregate (Epics 1.1-1.3, 3.1)

Support	Evidence
Gap	small-accountant-flow-proposal: AccountingSystem is 29-line shell, no Journal aggregate exists
Active	Issues #2099 (Accounting phase 2), #2106-#2109 (opening balances)
Dependency	Every other accountant workflow epic depends on company setup + journal
Growth	AccountBalance (2 PRs), ChartOfAccounts (14 PRs) — next growth aggregates
Risk	Must not expand God Aggregate — proposal explicitly routes new features to new aggregates

This trial tests whether the vault guides agents to build new aggregates correctly (with RestoreFromEvents tests, source ctors, proper isolation) rather than expanding ServerFinancialModel.

12. Vault Document Index

Scan-Generated (refreshed 2026-03-08)

Doc	Key Data
scan-data.json	27 aggregates, 249 commands, 443 events, 22 handlers, 80 RMs
aggregates.md	Sizes, Apply/Register counts, missing source ctors, god-aggregate flag
message-map.md	Command → handler routing, aggregate touch map
read-models.md	RM subscriptions with start<Tagg>() extraction, category/event-type streams
bounded-contexts.md	7 contexts, aggregate/handler/RM ownership
test-coverage.md	0/27 RestoreFromEvents, 24/27 idempotency, 25/27 state guards

god-aggregate.md	ServerFinancialModel: 3,557 lines, 87 Apply, expansion triggers
------------------	---

Agent-Generated Architecture (from codebase analysis)

Doc	Key Data
domain-model.md	Entity hierarchy, relationships, temporal model gaps
application-topology.md	4 hosts (WPF, Excel, Blazor, QBConnector), per-business DataStore
project-architecture.md	Layer diagram, dependency rules, package distribution
message-bus-topology.md	Two-bus architecture, envelope commands, NullBus replay safety
isolation-acl.md	Context hierarchy, read/write separation, ACL files
data-usage-and-storage.md	Per-category-stream amplification, write volumes, performance implications

GitHub-Extracted

Doc	Key Data
feature-provenance.md	229 features, 10 milestones, aggregate touch history
saga-catalog.md	7 workflow chains, all implicit, risk matrix, process manager infrastructure reference
defect-analysis.md	66 defects, context/RM hotspots, recurring patterns

New in v3

Doc	Key Data
small-accountant-flow-proposal.md	5 epics, 15 issues, 3 new aggregates, extends AccountingSystem
process-manager/Infrastructure/	ProcessAggregateBase, ProcessManagerBase, CategoryStreamWarmup, EventVersioning
process-manager/read-model-caching-reference/	IReadModelStore, ICheckpointStore, DapperReadModelStore, PostgresCheckpointStore

Planning Vault

Section	Count	Purpose
Facets	23	Product capabilities and technical patterns
Audiences	11	Stakeholder perspectives
Intersections	253	Facet × audience resonance matrix

The vault now has four data layers:

- 1. **What exists** (scan artifacts)
- 2. **What it looks like structurally** (architecture docs)
- 3. **Why it exists and where it breaks** (GitHub history)

4. **What's coming and what infrastructure is ready** (accountant workflow + process manager + RM caching)